

Numerical Simulation of Learning Dynamics in Multimodal Reinforcement Learning: A Derived and Numerically Verified Stability Law for Diversity-Preserving Training

CSE 402 Final Project Report · Section A (Subsection A2) · Group 04

Ahnaf Tahmid
ID 2105041 · Dept. of CSE, BUET
Dhaka, Bangladesh

Sadia Binte Sayed
ID 2105045 · Dept. of CSE, BUET
Dhaka, Bangladesh

Mohammad Raihan Rashid
ID 2105046 · Dept. of CSE, BUET
Dhaka, Bangladesh

Ruwad Naswan
ID 2105051 · Dept. of CSE, BUET
Dhaka, Bangladesh

Md. Mehedi Hasan
ID 2105052 · Dept. of CSE, BUET
Dhaka, Bangladesh

Abstract

Policies trained by expected-return maximization often collapse onto one outcome even when many outcomes are equally good. Our base paper, Sinha, Elango and Liu (2026), traces this to the probability multiplier in the policy gradient and removes it with inverse probability scaling (IPS). We generalize IPS with a tunable exponent α , $\tilde{r} = r/p^\alpha$, and use numerical analysis to derive a closed-form *stability law* for the resulting training dynamics: the stationary policy $p^* \propto r^{1/\alpha}$, the linear convergence rates, the largest stable learning rate, and a *universal survival limit* for finite groups of G rollouts, which for the paper’s clipped estimator reads $\alpha > \ln(r_1/r_2)/\ln \min\{G, 1/\varepsilon\}$. Every part is verified with methods implemented from scratch: Euler and RK4 integration, bisection, secant and Newton–Raphson root finding, least-squares fitting, a Monte Carlo training agent based on inverse-transform sampling, a 19,125-point parameter hypergrid with multilinear interpolation, exact binomial error analysis, golden-section search, and power/QR eigenvalue methods. The stationary law is reproduced to 7.5×10^{-15} , the rates to 1.7×10^{-5} and the stable step size to 2.6×10^{-4} ; the survival limit classifies every off-boundary hypergrid point correctly, and Monte Carlo training matches the predicted finite-group targets to within 0.013. The analysis also shows that the paper’s $\alpha = 1$ rule is exact up to the miss probability $(1-p)^G$, but loses an outcome whenever $G \leq r_1/r_2$. An interactive dashboard runs every model live. Code: <https://github.com/rwd51/alpha-ips-rl>.

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**: *Simulation evaluation*; • **Mathematics of computing** → **Numerical analysis**.

Keywords

reinforcement learning, mode collapse, inverse probability scaling, root finding, ODE integration, Monte Carlo simulation, stability analysis

1 Introduction and Base Paper Review

1.1 Background

Many reinforcement learning (RL) problems have several terminal outcomes of comparable quality: LLM post-training, program synthesis, molecular design. There the goal is to keep *all* good outcomes, not to find one. In practice, policies trained on the expected return $J = \sum_o p_o r(o)$ collapse onto a small subset of them, and the usual explanation is insufficient exploration or weak regularization. Flow-based methods such as GFlowNets avoid collapse by matching a target distribution, but need specialized objectives and auxiliary networks.

1.2 Base paper: Sinha, Elango and Liu (2026)

The base paper [3] argues that outcome-level mode collapse is a structural property of the objective itself, not an exploration failure.

Diagnosis. For a softmax policy over outcomes under gradient flow on J , its Theorem 3.1 gives $\frac{d}{dt} \log \frac{p_i}{p_j} = p_i a_i - p_j a_j$ with advantage $a_i = r_i - \bar{r}$. The update of each outcome is multiplied by its own probability, so an outcome that is slightly ahead receives larger updates and pulls further ahead: a rich-get-richer loop that ends at $p_i = 1$, whatever the exploration scheme or entropy bonus.

Fix. Inverse probability scaling (IPS) replaces the reward by $\tilde{r}(o) = r(o)/p(o)$, with p held fixed when differentiating (stop-gradient). The flow becomes $\dot{z}_i = r_i - p_i \sum_k r_k$ (its Theorem 4.1), whose stationary point is the reward-proportional policy $p_i^* = r_i / \sum_k r_k$, and a potential function proves global convergence. With a KL term, IPS interpolates between this policy and the reference policy instead of fighting collapse.

Algorithm. IPS-GRPO plugs the correction into GRPO, a group-based policy-gradient method: each group of G rollouts estimates the outcome frequency $\hat{p}(o)$ by counting, and rescales the reward as $\tilde{r} = r / \max\{\hat{p}(o), \varepsilon\}$ (its Eq. 9), with the clip ε preventing extreme weights.

Evidence. On a 2-D hyper-grid the ℓ_1 distance to the target distribution falls from 2.8×10^{-2} (GRPO and FlowRL) to 3.0×10^{-3} . On the HypoSpace benchmark (Gemma-3 4B with LoRA) the recovery rate of admissible answers rises from 16.0% to 43.9% (causal inference), 31.6% to 90.0% (3-D voxels) and 9.5% to 60.7% (Boolean expressions). In molecular design it raises the yield of REINVENT (SYNTH task, threshold 0.80) from 5695 to 7371 unique molecules.

An ablation over G and ε (its Table 4) is read as a bias–variance trade-off. The paper lists as limitations the noisy finite-group estimate $\hat{p}$, the need for clipping, and the restriction to terminal rewards and non-actor-critic methods.

1.3 What the base paper leaves open, and our objective

The paper gives one fixed correction: full collapse ($\alpha = 0$) or strict proportionality ($\alpha = 1$), so diversity cannot be tuned. It also leaves open the questions a practitioner faces: how large must the group be, what does the clip do, which learning rate is stable, and how fast does training converge? Its claim that collapse happens “even with equal rewards” is argued from a deterministic flow whose drift is exactly zero at a tie.

Our objective is to *derive, with the methods of numerical analysis, a formula that says when this training is stable and keeps every good outcome, and to verify that formula numerically*. Box 1 states the result, and Figure 1 shows how the work fits together.

Contributions.

- A generalized update $\tilde{r} = r/p^\alpha$ whose stationary law $p^\star \propto r^{1/\alpha}$ makes α a continuous diversity knob.
- A **stability law** (Box 1): convergence rates, explicit step-size limits, the finite-group survival limit, and its universal form for any weight estimator (Section 2.2).
- A proof that $\alpha = 0$ is a singular limit: at an exact tie the deterministic flow does not move, and convergence turns from exponential to algebraic.
- Numerical verification of every formula by five experiments and a Monte Carlo agent, presented as 18 validation cases that each pair a prediction with its graphs and the measured result (Section 4).
- An interactive simulation dashboard with 19 live instruments that reproduce the experiments in the browser (Section 3.2).

2 Methodology

2.1 Simulation model: the outcome-selection bandit

In plain words: training is reduced to its core, a list of answers whose probabilities move toward higher reward.

Following the base paper’s own abstraction, we remove states and exploration and keep only the objective. A policy chooses one of K outcomes with fixed rewards $r_1, \dots, r_K > 0$. It stores one score (logit) z_i per outcome and turns the scores into probabilities with the softmax, $p_i = e^{z_i} / \sum_k e^{z_k}$. Training is gradient ascent on the expected reward $J = \sum_i p_i r_i$; with a very small step it becomes a differential equation in time (a *gradient flow*, the idealized learning dynamics of [3]):

$$\dot{z}_i = \frac{\partial J}{\partial z_i} = p_i (r_i - \bar{r}), \quad \bar{r} = \sum_k p_k r_k. \quad (1)$$

Each answer moves in proportion to how much better than average it is ($r_i - \bar{r}$), multiplied by its own probability p_i . That factor p_i is the *probability multiplier* that drives collapse: a popular answer gets

Table 1: Notation: every symbol used in this report and what it means.

Symbol	Meaning
K	number of possible answers (outcomes)
r_i	reward of answer i ; $\rho = r_1/r_2$ is a reward ratio
z_i	score (logit) of answer i , the quantity training changes
p_i	probability of answer i , $p = \text{softmax}(z)$: $p_i = e^{z_i} / \sum_k e^{z_k}$
$\dot{z}_i$	rate of change of z_i during training (dz_i/dt)
J	expected reward $\sum_i p_i r_i$, what ordinary training maximizes
$\bar{r}, a_i$	average reward $\sum_k p_k r_k$; advantage $a_i = r_i - \bar{r}$
α	diversity exponent: reward is divided by p^α
$p^\star$	stationary policy: where training ends ($\dot{z} = 0$)
λ	convergence rate: error shrinks like $e^{-\lambda t}$
$J^\star$	Jacobian at $p^\star$: how $\dot{z}$ reacts to small changes in z
h	step size (learning rate) of one update
G	group size: samples drawn per update
$n_i, \hat{p}_i$	count of answer i in a group; its frequency n_i/G
ε	clip: $\hat{p}$ is never taken below ε
$\omega(n)$	weight rule: weight given to an answer seen n times
$w_G(p)$	effective weight: average weight a finite group really applies
S	common level that all surviving answers balance at
m, α_c	survival margin; smallest α that keeps an answer
D	distortion: relative error of w_G against the ideal $p^{-\alpha}$
β, κ	moving-average rate; its rate per unit time, β/h

bigger updates and grows even more popular. The whole state is a list of K numbers, so no neural network, GPU or dataset is needed.

IPS divides each reward by the answer’s probability, $r(o)/p_o$, which cancels the multiplier. Our extension, from the project proposal, makes the strength of that correction tunable with an exponent:

$$\tilde{r}_\alpha(o) = \frac{r(o)}{p_o^\alpha}, \quad (2)$$

where $\alpha = 0$ is the standard objective and $\alpha = 1$ is IPS.

2.2 Derivation of the stability law

Box 1 states the result; this subsection derives each part. Every equation is tested numerically in Section 4. Each part starts with a one-sentence summary for readers who want to skip the algebra.

2.2.1 The α -generalized gradient flow. *In plain words:* we work out how every score changes during training when rewards are divided by p^α ; the result is Eq. (3), used everywhere below.

Following the paper’s proof technique for IPS, the probabilities inside the weight are held fixed while we differentiate (a *stop-gradient*, $\bar{p} = \text{stopgrad}(p)$): the weight $w_k = r_k / \bar{p}_k^\alpha$ is treated as a constant, and only the outer p_k in $J_\alpha(z) = \sum_k p_k(z) w_k$ is differentiated. The softmax derivative is $\partial p_k / \partial z_i = p_k(\delta_{ik} - p_i)$, where δ_{ik} is 1 if $i = k$ and 0 otherwise, so

$$\frac{\partial J_\alpha}{\partial z_i} = \sum_k w_k p_k (\delta_{ik} - p_i) = p_i w_i - p_i \sum_k p_k w_k.$$

Since $\bar{p} = p$ numerically at the instant of evaluation,

$$\dot{z}_i = r_i p_i^{1-\alpha} - p_i \sum_k r_k p_k^{1-\alpha} \quad (3)$$

At $\alpha = 0$ this is exactly (1), the paper’s Eq. (2). At $\alpha = 1$ it is $\dot{z}_i = r_i - p_i R$, the paper’s IPS Eq. (4).

Box 1 – The α -IPS stability law (derived in Sec. 2.2, verified in Sec. 4)

For rewards $r_1 \geq \dots \geq r_K > 0$, exponent $\alpha > 0$, group size G and clip ε :

(S1) **Fixed point.** $p_i^* = \frac{r_i^{1/\alpha}}{\sum_k r_k^{1/\alpha}}$.

(S2) **Rate.** $\|p(t) - p^*\| \sim e^{-\lambda_{\min} t}$ with $\lambda_j = \alpha \|r\|_{1/\alpha} \mu_j$, where μ_j are the nonzero eigenvalues of $\text{diag}(p^*) - p^* p^{*\top}$ and $\|r\|_{1/\alpha} = (\sum_k r_k^{1/\alpha})^\alpha$.

(S3) **Stable step size.** An explicit update is stable iff $h\lambda_{\max} < 2$ (Euler) or $h\lambda_{\max} < 2.7853$ (RK4).

(S5) **Estimator exactness.** The dynamics feel the distortion $D = \alpha(\alpha - 1)(1 - p)/(2Gp) + O(G^{-2})$ of the clipped rule. At $\alpha = 1$ it is exactly $D = -(1 - p)^G$.

(S4) **Universal survival limit.** With finite groups of G rollouts, outcome 2 survives iff

$$\frac{\omega(1)}{\omega(G)} > \frac{r_1}{r_2}$$

which for the paper's clipped rule $\omega = \max\{n/G, \varepsilon\}^{-\alpha}$ becomes

$$\alpha > \alpha_c = \frac{\ln(r_1/r_2)}{\ln \min\{G, 1/\varepsilon\}}$$

Equivalently, $G > (r_1/r_2)^{1/\alpha}$ and $\varepsilon < (r_2/r_1)^{1/\alpha}$.

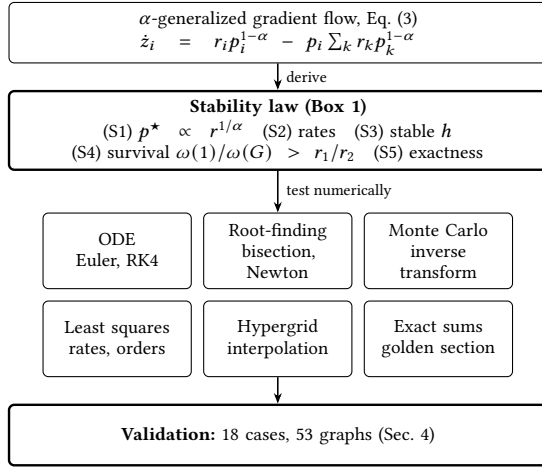


Figure 1: Project overview. A closed-form stability law is derived from the generalized flow, then every part is tested with course methods implemented from scratch.

2.2.2 Stationary law: the diversity knob. In plain words: training stops when every score stops changing, and that happens when each answer's probability is proportional to its reward raised to the power $1/\alpha$.

Write (3) as $\dot{z}_i = p_i (r_i p_i^{-\alpha} - S_\alpha)$ with $S_\alpha = \sum_k r_k p_k^{1-\alpha}$ (one number shared by all answers). At a rest point where every answer is still alive ($p_i > 0$) every bracket must vanish, so $r_i p_i^{-\alpha} = S_\alpha$ for all i , i.e. $p_i^\alpha \propto r_i$:

$$p_i^* \propto r_i^{1/\alpha} \quad (4)$$

So α is a *diversity knob*. As $\alpha \rightarrow 0$ all probability goes to the best answer (collapse); $\alpha = 1$ is proportional to reward, as in the base paper; $\alpha > 1$ flattens toward an even split. For $r = (4, 1)$ the split is 94/6, 80/20 and 67/33 at $\alpha = 0.5, 1, 2$.

2.2.3 Linearization, rates and step-size stability. In plain words: near the end point, the remaining error shrinks by a fixed factor per unit time; that rate λ tells how fast training converges and how large a learning rate is safe.

Linearization means following a very small deviation from the end point, for which the equations become linear and can be solved exactly. Perturb around $z^* = \log p^*$. With $C = \text{diag}(p) - pp^\top$ we have $\delta p = C \delta z$. At p^* every r_i/p_i^{α} equals $c = \|r\|_{1/\alpha}$, and $S^* = c$. Perturbing each term of (3): $\delta(r_i p_i^{1-\alpha}) = (1 - \alpha)c \delta p_i$, $\delta S = (1 - \alpha)c \sum_k \delta p_k = 0$, and $\delta(p_i S) = c \delta p_i$. Hence $\delta \dot{z}_i = -\alpha c \delta p_i$ and

$$J^* = -\alpha \|r\|_{1/\alpha} (\text{diag}(p^*) - p^* p^{*\top}). \quad (5)$$

J^* is the Jacobian (matrix of derivatives) at the end point. Its eigenvalues are the decay rates of independent error patterns. One eigenvalue is zero, for the pattern that adds the same number to every score, which does not change p ; the other $K - 1$ are the rates λ_j of (S2). For $K = 2$, $\lambda = 2\alpha \|r\|_{1/\alpha} p^* q^*$ ($r = (4, 1)$, $\alpha = 1$: $\lambda = 1.6$). A numerical method is *stable* if each step shrinks the error instead of growing it. On the test equation $\dot{x} = -\lambda x$ one Euler step multiplies the error by $1 - h\lambda$, so it is stable iff $|1 - h\lambda| < 1$; one RK4 step multiplies it by $R(-h\lambda)$ with $R(z) = 1 + z + z^2/2 + z^3/6 + z^4/24$. The real-axis limit is the root of $x^3 - 4x^2 + 12x - 24 = 0$, $x = 2.7853$, which gives (S3).

Two regimes follow. **Large α is stiff** (some rates are very large, so only tiny steps are safe): $\|r\|_{1/\alpha}$ grows like K^α , so $\lambda_{\max}$ and hence the stable learning rate change exponentially in α . **Small α is slow**: for $K = 2$, $\lambda \approx 2\alpha r_1(r_2/r_1)^{1/\alpha}$, which vanishes faster than any power of α .

2.2.4 $\alpha = 0$ is a singular limit. In plain words: ordinary training behaves differently from every $\alpha > 0$: with exactly equal rewards it does not move at all, and otherwise it collapses slowly, like $1/t$, not exponentially.

Exact tie. If $r_1 = \dots = r_K$, then $r_i - \bar{r} \equiv 0$ for every p , so (1) gives $\dot{z} \equiv 0$. The deterministic flow does not move. The paper's claim that collapse happens "even with equal rewards" therefore describes the *stochastic* process, where the finite group never reports the exact mean reward (Section 2.3.4).

Algebraic convergence. For $K = 2$, $u = z_1 - z_2$ and $\Delta = r_1 - r_2 > 0$, (3) at $\alpha = 0$ gives $\dot{u} = 2pq\Delta = \Delta/(1 + \cosh u)$, which integrates exactly to

$$(u + \sinh u) - (u_0 + \sinh u_0) = \Delta t. \quad (6)$$

For large u , $p_2 \approx 1/(2\Delta t)$: a power law with no exponential rate. From $u_0 = 0.1$ to $p_1 = 0.99$ it predicts $t = 53.89/\Delta$.

Methodology at a glance (for readers who want the idea, not the algebra)

- (1) **The model.** A policy chooses one of K answers. Each answer has a fixed reward and a probability; training nudges the probabilities toward answers with more reward. This is the base paper's own simplified test bed.
- (2) **The change we study.** Each reward is divided by the answer's probability raised to a power α . With $\alpha = 0$ we get ordinary training, which collapses onto one answer; with $\alpha = 1$ the base paper's fix; other values tune how evenly the answers are kept.

Reading guide. Readers interested only in the results can read Box 1 and the *In plain words* sentence that opens each part of Section 2.2, then go to Section 4. Table 1 explains every symbol.

- (3) **What we derive (Section 2.2).** Formulas for where training ends, how fast it gets there, the largest safe learning rate, and how large a sample group must be so that no good answer dies out (Box 1).
- (4) **How we check it (Section 2.3).** Each formula is compared with a course method: simulating training (Euler, RK4), solving for the end point directly (bisection, Newton), simulating real sampling (Monte Carlo), and exact error sums.

2.2.5 Finite groups: the universal survival limit. *In plain words:* with only G samples, a rare answer can be boosted at most about G^α times; if its reward is too far below the best one, that boost is not enough and it dies out.

A real algorithm never knows p . It sees a group of G rollouts with counts $n_i \sim \text{Binomial}(G, p_i)$ and uses $\hat{p}_i = n_i/G$. The sampled update is

$$\hat{g}_i = r_i \hat{p}_i \omega(n_i) - p_i \sum_k r_k \hat{p}_k \omega(n_k), \quad (7)$$

with the paper's weight rule $\omega(n) = \max\{n/G, \varepsilon\}^{-\alpha}$. To predict long-run behaviour we use the *mean field*: the average of this random update over all possible groups ($\mathbb{E}$ denotes an average). Averaging replaces $p_i^{1-\alpha}$ by $\phi_G(p_i) = \mathbb{E}[\hat{p}_i \omega(n)]$. The size-biasing identity $n \binom{G}{n} p^n (1-p)^{G-n} = G p \binom{G-1}{n-1} p^{n-1} (1-p)^{G-n}$ gives the *effective weight*

$$w_G(p) = \frac{\phi_G(p)}{p} = \mathbb{E}[\omega(1+B)], \quad (8)$$

with $B \sim \text{Bin}(G-1, p)$, the number of times the answer appears among the other $G-1$ samples. This is the weight given to a group already known to contain one copy of the outcome. It is well defined at $p=0$ and strictly decreasing, with endpoints $w_G(0) = \omega(1)$ and $w_G(1) = \omega(G)$. For the clipped rule, $w_G(0) = \min\{G, 1/\varepsilon\}^\alpha$: the largest boost a finite group can give, because a rare outcome is seen at most once, at frequency $1/G$.

THEOREM 2.1 (UNIVERSAL SURVIVAL LIMIT). *Let $K = 2$ and $\rho = r_1/r_2 \geq 1$. For any weight rule ω , the mean field has an interior rest point (the minority survives) iff*

$$\frac{\omega(1)}{\omega(G)} > \rho.$$

PROOF. Rest points solve $f(p_1) = r_1 w_G(p_1) - r_2 w_G(1-p_1) = 0$. At $p_1 = \frac{1}{2}$, $f = (r_1 - r_2) w_G(\frac{1}{2}) > 0$. At $p_1 = 1$, $f = r_1 \omega(G) - r_2 \omega(1)$. Since w_G is monotone, a root in $(\frac{1}{2}, 1)$ exists iff $f(1) < 0$. $\square$

In words: a minority answer survives exactly when the largest weight a group can give it, $\omega(1)$, beats the weight of a full group, $\omega(G)$, by more than the reward ratio. For the clipped rule this gives (S4). We define the *phase margin* $m = \alpha \ln \min\{G, 1/\varepsilon\} - \ln \rho$, so that $m > 0$ means survival. Two practical readings follow. (i) Lowering ε below $1/G$ changes nothing, because every nonzero $\hat{p}$ is already

at least $1/G$. (ii) Even the paper's $\alpha = 1$ needs $G > r_1/r_2$: with $r = (4, 1)$, IPS-GRPO at $G = 4$ loses the weaker outcome.

COROLLARY 2.2 (ANY ESTIMATOR). *Laplace smoothing $\omega = ((n + \lambda)/(G + K\lambda))^{-\alpha}$ has range $((G + \lambda)/(1 + \lambda))^\alpha < G^\alpha$, independent of K , so it always raises α_c . Naive split-half Richardson gives a singleton the negative weight $\omega_R(1) = 2G^\alpha - \frac{1}{2}(G/2)^\alpha - \frac{1}{2}\varepsilon^{-\alpha}$, so the minority dies unconditionally.*

2.2.6 General K : extinction thresholds. *In plain words:* with many answers, every survivor ends at a common level S ; weak answers need a larger α than the two-answer formula says, and some cannot be saved by any α when G is small.

For any K a rest point satisfies three conditions (the KKT-type conditions of optimization): surviving answers balance at a common level, dead answers cannot re-enter, and probabilities sum to one: $r_i w_G(p_i) = S$ if $p_i > 0$, $r_i w_G(0) \leq S$ if $p_i = 0$, and $\sum_i p_i = 1$. Because w_G is monotone, $p_i(S) = w_G^{-1}(S/r_i)$ is unique, and $T(S) = \sum_i p_i(S) - 1$ has one root on $[r_{\max}, r_{\max} w_G(0)]$. That gives a nested root problem with a guaranteed bracket. Outcome j is kept iff $h_j(\alpha) = \ln(r_j \min\{G, 1/\varepsilon\}^\alpha) - \ln S^*(\alpha) > 0$. For the runner-up $S^* = r_1$, which recovers (S4) exactly. Weaker outcomes need a strictly larger α , because stronger outcomes still share mass and push S^* above r_1 . As $\alpha \rightarrow \infty$ (with $G < 1/\varepsilon$), outcome j survives iff $\sum_{i \in A, i \neq j} (r_j/r_i)^{1/(G-1)} > |A| - 2$, where $A = \{i : r_i \geq r_j\}$. For $K > 2$ this can fail, and then *no* α saves outcome j .

2.2.7 Estimator error and what the dynamics see. *In plain words:* the sample frequency $\hat{p}$ is a noisy guess of p ; we compute how wrong the weight $\hat{p}^{-\alpha}$ is, which part of that error actually changes where training ends, and what a moving average buys.

Delta method (a Taylor expansion of a function of a noisy estimate). *Bias* is the average error, *spread* the standard deviation, both relative to the true weight $p^{-\alpha}$. Expanding $f(\hat{p}) = \hat{p}^{-\alpha}$ about p (the course's truncation-error expansion with $\hat{p} - p$ as the step) gives the relative bias and spread

$$b = \frac{\alpha(\alpha+1)(1-p)}{2Gp}, \quad s = \alpha \sqrt{\frac{1-p}{Gp}}. \quad (9)$$

The expansion parameter is Gp , the expected number of hits, not G .

Optimal clip. The mean squared error (MSE) adds bias² and spread². On each interval where $\Pr[n < G\varepsilon]$ is constant, $d \text{MSE}/d\varepsilon = -2\alpha \Pi \varepsilon^{-\alpha-1} (\varepsilon^{-\alpha} - p^{-\alpha})$, so the MSE falls for $\varepsilon < p$ and rises for

Table 2: Course methods and where they enter the project.

Method	Used for
Euler, RK4	integrating the flow; order and stability
Bisection	roots, critical h , extinction α_c
Secant, Newton–Raphson	stationary point, nested mean field
Least squares	convergence rates, power laws, orders
Inverse-transform sampling	Monte Carlo agent
Multilinear interpolation	hypergrid surrogate
Taylor / delta method	estimator bias and variance
Richardson extrapolation	bias correction in $1/G$
Golden-section search	optimal ε , EMA β
Power method, QR algorithm	spectrum of the Jacobian

$\varepsilon > p$: **exactly** $\varepsilon^* = p$. One global ε cannot suit a majority and a minority outcome at the same time.

Dynamics-level distortion. The dynamics see w_G of (8), not $\mathbb{E}[\omega(n)]$. With $X = (1+B)/G$, $\mathbb{E}X = p + (1-p)/G$: the guaranteed hit shifts the argument up and partly cancels the convexity bias,

$$D = w_G p^\alpha - 1 = \frac{\alpha(\alpha-1)(1-p)}{2Gp} + O(G^{-2}). \quad (10)$$

At $\alpha = 1$ the sum is exact: $\mathbb{E}[1/(1+B)] = (1-q^G)/(Gp)$, so

$$w_G(p) = \frac{1 - (1-p)^G}{p}, \quad D = -(1-p)^G \quad (11)$$

The paper’s rule is therefore right to within the probability that the group misses the outcome, an exponentially small error. A bias-corrected (Richardson) rule has $D = -\alpha(1-p)/(Gp)$ and breaks this cancellation. For $\alpha \neq 1$ the offset rule $\max\{(n-c)/(G-c), \varepsilon\}^{-\alpha}$ with $c^* = (1-\alpha)/2$ removes the $O(1/G)$ term.

Moving average. An exponential moving average (EMA) of the frequency over training steps, $\bar{p}_t = (1-\beta)\bar{p}_{t-1} + \beta\hat{p}_t$ has $\text{Var}(\bar{p}) = \frac{\beta}{2-\beta}\text{Var}(\hat{p})$, i.e. $G_{\text{eff}} = G(2-\beta)/\beta$, and a lag $(1-\beta)\delta/\beta$ behind a policy drifting by δ per step. With $\kappa = \beta/h$, linearizing the coupled $(z, \bar{p})$ system turns (S2) into a damped oscillator

$$x'' + \kappa x' + \kappa \lambda_j x = 0, \quad (12)$$

which rings for $\kappa < 4\lambda_j$ and converges fastest, at rate $2\lambda_j$, when $\kappa = 4\lambda_j$.

2.3 Numerical methods

Table 2 maps each course method to where it is used. All of them are implemented from scratch; library routines (SciPy’s DOP853 integrator [5] and `numpy.linalg`) serve only as independent references.

2.3.1 ODE integration. The flow (3) is integrated with our own Euler and RK4. The production step is $h = 0.05$ (Exp. 1) or $h = 0.005$ (Exp. 2). Collapse times are located by event detection on $p_1 = 0.99$. The reference solution is scipy DOP853 with `rtol = 10-12`, used only to measure error. Observed orders are least-squares slopes of log error against log h , fitted above the round-off floor.

2.3.2 Root-finding. The stationarity condition is solved in three equivalent forms: *drift* $F_i = \dot{z}_i$; *balance* $F_i = r_i p_i^{-\alpha} - r_K p_K^{-\alpha}$ (the drift divided by p_i); and *log* $F_i = \ln(r_i/r_K) - \alpha(z_i - z_K)$, which is linear in z . Because $J1 = 0$, Newton fixes the gauge $z_K = 0$ and solves for $K-1$ unknowns. Bisection, secant and Newton (plain, with Armijo backtracking, and safeguarded by a bracket) are compared by iteration count, empirical order and basin of convergence. The finite- G mean field for any K is solved by nested root-finding: an inner solve of $w_G(p_i) = S/r_i$ and an outer solve of $\sum_i p_i(S) = 1$. The exact derivative $\frac{d}{dp} \mathbb{E}[f(1+B)] = (G-1) \mathbb{E}_{\text{Bin}(G-2,p)} [f(2+B) - f(1+B)]$ supplies Newton. Critical step sizes and extinction exponents α_c are found by bisection on the actual nonlinear iteration.

2.3.3 Least-squares fitting. Rates are the slope of $\ln\|p(t) - p^*\|_1$ against t inside the window $[10^{-12}, 10^{-6}]$: past the nonlinear transient and above round-off. Power laws (collapse time against Δ and G , algebraic tails, bias orders) are straight-line fits in log–log coordinates.

2.3.4 Monte Carlo simulation. The Monte Carlo agent is the realistic, stochastic counterpart of the flow. It does what GRPO does: it samples a group, estimates $\hat{p}$ from counts, and takes one gradient step. Algorithm 1 is one training step, run for S independent seeds at once (`src/dynamics.py`).

Algorithm 1 — one Monte Carlo training step

Input: logits $z \in \mathbb{R}^{S \times K}$, rewards r , exponent α , group size G , clip ε , learning rate h .

- (1) $p \leftarrow \text{softmax}(z)$, row by row.
- (2) CDF $c_k = \sum_{j \leq k} p_j$, with $c_K \leftarrow 1$ to guard against round-off.
- (3) Draw $u \sim U(0, 1)^{S \times G}$; each rollout’s outcome is $\# \{k : c_k < u\}$ (inverse-transform sampling).
- (4) Count outcomes: n_i , and $\hat{p}_i = n_i/G$.
- (5) Weights $w_i = r_i \hat{p}_i \max\{\hat{p}_i, \varepsilon\}^{-\alpha}$ ($w_i = 0$ if $n_i = 0$).
- (6) Update direction $\hat{g}_i = w_i - p_i \sum_k w_k$, i.e. Eq. (7).
- (7) $z \leftarrow z + h \hat{g}$: one noisy Euler (SGD) step, not RK4, since $\hat{g}$ is random.

Worked example: $\alpha = 1$. With $\varepsilon \leq 1/G$, step 5 gives $w_i = r_i$ whenever outcome i appears in the group and $w_i = 0$ otherwise. So $\hat{g}_i = r_i \mathbf{1}[n_i > 0] - p_i \sum_k r_k \mathbf{1}[n_k > 0]$: the only randomness is *which* outcomes the group contains. Taking expectations, $\mathbb{E} \mathbf{1}[n_i > 0] = 1 - (1-p_i)^G$, which is exactly the effective weight $p_i w_G(p_i)$ of Eq. (11). The simulated agent and the finite- G theory describe the same object. Once every outcome appears in the group, the sampled step equals the IPS flow $\dot{z}_i = r_i - p_i \sum_k r_k$.

What is measured. *Collapse* is declared when $\max_i p_i > 0.95$, and the step is recorded. *Long-run policies* are averaged over the second half of the run to remove the transient and average out the noise. *Diversity* is the normalized entropy $H(p)/\ln K$. *Accuracy* is the ℓ_1 distance to the ideal p^* , the metric of the base paper’s Table 1.

How it is validated. The expected update $\mathbb{E}[\hat{g}]$ is a finite binomial sum, so the Monte Carlo runs are compared against the exact finite- G mean field (8), not just against the $G \rightarrow \infty$ law. The residual gap is then decomposed into its two known sources: sampling noise, which shrinks with seeds, and finite- h bias, which shrinks with h (Case 11). For estimator statistics (Exp. 5), exact binomial sums

are the ground truth, and Monte Carlo checks them with z-scores built from their own exact standard errors. For the variance this uses $\sqrt{(\mu_4 - \sigma^4)/N}$, because the clipped weight is heavy-tailed. Every experiment uses fixed NumPy seeds, logs its trajectories and statistics to CSV/JSON, and reproduces every CSV byte for byte. Table 4 lists every Monte Carlo run.

2.3.5 Hypergrid and interpolation. The $K = 2$ mean field is solved on a four-dimensional tensor grid $(\alpha, G, \varepsilon, \rho)$ by a batched 64-iteration bisection. Because the batched solver picks the collapsed or interior branch from the margin m itself, every point is also re-solved by the scalar solver of Experiment 2, which decides collapse from the sign of the balance function and never uses m . A 17^3 grid in $(\log \alpha, \log \rho, \log \varepsilon)$ at $G = 16$ is then used as a multilinear surrogate, $\hat{f}(\mathbf{x}) = \sum_{\mathbf{c} \in \{0,1\}^D} f(\mathbf{x}_{\mathbf{c}}) \prod_d t_d^{c_d} (1 - t_d)^{1-c_d}$, and audited against 800 seeded off-grid direct solves. The response has derivative kinks on the extinction surface $m = 0$ and at every clip threshold $\varepsilon = k/G$, so each holdout point is attributed to the cell its interpolation uses, and errors are reported separately for kink-free and kinked cells.

2.3.6 Estimator analysis. Every bias and variance is an exact sum $\sum_n \binom{G}{n} p^n (1-p)^{G-n} \omega(n)$. Minima are found by golden-section search, audited by a dense scan with unimodality asserted. Its accuracy is certified against two closed forms ($\varepsilon^* = p$ and the add- λ optimum $\lambda^* = Kp(1-p)/(1-Kp)^2$). The eigenvalues μ_j in (S2) and (12) come from our power method with Hotelling deflation and an unshifted QR algorithm, whose Gram-Schmidt step has a rank-deficiency guard.

2.3.7 Round-off safeguards. $p^{-\alpha}$ overflows for tiny p , and the absolute bias is a difference of two huge, nearly equal numbers. Every error is therefore formed as $\mathbb{E}[\omega p^\alpha] - 1$, a sum of $O(1)$ terms, and powers are evaluated as $\exp(-\alpha \ln x)$. In (11), $1 - (1-p)^G$ is evaluated as $-\expm1(G \log p(-p))$ below $p = 10^{-6}$.

3 Implementation and Experiments

3.1 Implementation

The code is written in Python with NumPy [1], SciPy [5] and Matplotlib [2], and runs on a CPU; no GPU, neural network or external RL library is used, because the whole state is a K -vector of logits. The repository [4] is organized as:

- `src/`: the shared numerical library, written from scratch: the flow, Jacobian and Monte Carlo sampler (`dynamics.py`), Euler/RK4 and event detection (`integrators.py`), bisection, secant, Newton and multivariate Newton (`rootfinding.py`, `newton.py`), the three stationarity forms (`stationarity.py`), the finite-group mean field and nested solver (`finite_group*.py`, `meanfield_rule.py`), tensor grids and multilinear interpolation (`hypergrid.py`), weight rules and their exact statistics (`estimators.py`), golden-section search (`optimize1d.py`), power method and QR (`eigen.py`), moving averages (`ema.py`), and least squares (`fitting.py`).
- `experiments/`: one script per experiment, writing figures to `results/exp<N>_<name>/figures/` and CSV/JSON data next to them.
- `main.ipynb`: a notebook that runs any experiment and shows its figures (it also runs on Kaggle).

- `tests/`: 75 unit tests (55 for the estimators, 20 for the hypergrid), including comparisons with SciPy references.
- `dashboard/`: the interactive simulation dashboard (Section 3.2).

All random numbers come from fixed NumPy seeds, so every run is reproducible. The final runs used Python 3.11, NumPy 1.26, SciPy 1.14 and Matplotlib 3.8. Appendix B lists the commands that download the code and regenerate every result.

3.2 Interactive simulation dashboard

We also built an interactive dashboard (`dashboard/`) that runs every model of this report live in the browser: moving a slider for α , G , ε or h shows its effect on training at once. It is a static web page (TypeScript, React, three.js for 3-D) whose numerical core is a direct port of our Python library, so its readouts match the batch experiments (for example, 56 bisection iterations and Newton order 2.00 in the root-finder race). It has 19 instruments, three to four per experiment; Figure 2 shows six of them. We used it in the project defence.

3.3 Benchmarks, ground truth and metrics

There is no external dataset: the test bed is the outcome-selection bandit of Section 2.1. Every numerical result is checked against an independent reference: (i) the closed forms of Section 2.2 (p^* , the exact $\alpha = 0$ solution, the linear rates, the exact $\alpha = 1$ weight); (ii) a high-accuracy library integrator (DOP853, `rtol` = 10^{-12}) and `numpy.linalg` eigenvalues; (iii) a second, independent solver for the same quantity (scalar vs. batched bisection, nested bisection vs. safeguarded Newton); (iv) exact binomial sums for every estimator statistic; and (v) Monte Carlo training runs for every mean-field prediction. The metrics are the ℓ_1 distance to the target distribution (the base paper’s metric), the normalized entropy $H(p)/\ln K$, empirical orders of convergence from least-squares slopes, iteration counts, residuals, condition numbers, and z-scores.

3.4 Experiments and parameter choices

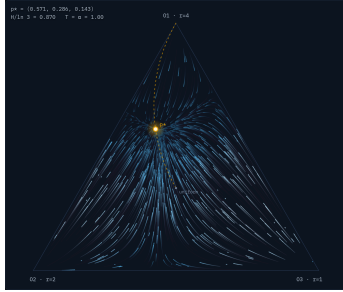
Table 3 summarizes the five experiments. Table 4 lists every Monte Carlo run. Parameter choices follow the theory: $\varepsilon = 10^{-3}$ lies below $1/G$ for every group size used, so the clip never fires unless it is the object of study; the step sizes $h = 0.05$ and 0.005 lie inside the stability limit (S3); $r = (4, 1)$ reproduces the proposal’s 80/20 split at $\alpha = 1$; and $r = (5, 4, 3, 2, 1)$ is a graded multi-outcome profile in which the weakest outcome is the hardest to keep.

4 Results and Discussion

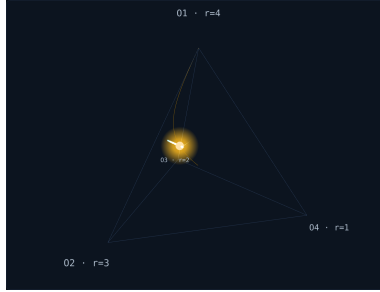
We test the stability law in 18 validation cases, one full-width figure each (Cases 1–18). Every case figure reads top to bottom: three fields state the **Prediction** from Section 2.2, the numerical **Test** with its data, and the measured **Result**; then three graphs follow, each with a one-line reading of what it shows. Theory is drawn as lines and numerics as markers throughout. The graphs are the experiments’ own figures, cut into single panels without change (`report/make_panels.py`). The caption names the formula under test and the experiment that produced the graphs; cases marked

Table 3: The five experiments: the question each answers, its setup and parameters, and the validation cases that report it. Runtimes are for a laptop CPU.

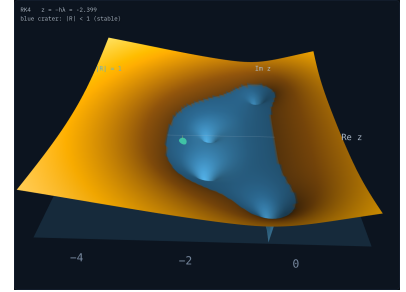
Experiment	Question	Setup and parameters	Cases	Runtime
1. Collapse baseline ($\alpha = 0$)	Does the standard objective collapse, and how fast?	$K = 2, 5$; RK4, $h = 0.05$; 12 gaps $\Delta \in [0.02, 4]$; Monte Carlo $G = 16$, $h = 0.5$, 36/200 seeds, $G \in \{4, \dots, 64\}$ with 80 seeds each	1, 2, 6	< 1 min
2. Diversity exponent	Does training end at $p^* \propto r^{1/\alpha}$? How fast? Which step is stable? What does finite G change?	$r = (4, 1), (2, 1), (1.25, 1), (5, 4, 3, 2, 1)$; $\alpha \in [0.3, 5]$; RK4, $h = 0.005$, $T = 200$; rate window $[10^{-12}, 10^{-6}]$; 30-halving bisection on h ; Monte Carlo $G \in \{2, \dots, 128\}$, $h = 0.05$, 3×10^4 steps, 16 seeds	3, 4, 5, 9	≈ 2.5 min
3. Root finding	Can $\dot{z} = 0$ be solved directly, does the formulation matter, and where is the finite- G target?	$r = (4, 1)$, $\alpha = 1$, $u_0 = 0$; basin $\alpha \in [0.2, 4] \times u_0 \in [-15, 15]$; $K = 5$ with 200 random starts $z \sim N(0, 3^2)$; nested solver $G \in \{2, \dots, 256\}$; Monte Carlo $h = 0.01$, 10^5 steps, 16 seeds	7, 8, 11	≈ 6 min
4. Parameter hypergrid	Does the survival limit hold jointly over $(\alpha, G, \varepsilon, \rho)$? Can interpolation replace the solver?	$51 \times 15 \times 5 \times 5 = 19,125$ points; $K = 5$ atlas $45 \alpha \times 17 G$; minimum- G search for $K \leq 10$, spreads 1.25–32; 17^3 grid at $G = 16$, 800 holdout points, refinement $5 \rightarrow 9 \rightarrow 17$	10, 12, 13	≈ 7 min
5. Estimator error	How far is $\hat{p}$ from p , which device trades bias for variance best, and what reaches the dynamics?	$G = 16$, $\alpha = 1$, $\varepsilon = 10^{-3}$ by default; exact binomial sums; 400,000 Monte Carlo groups per p ; G up to 4096; $K = 5$ training with 32 seeds; 60,000 EMA chains	14–18	≈ 10 min



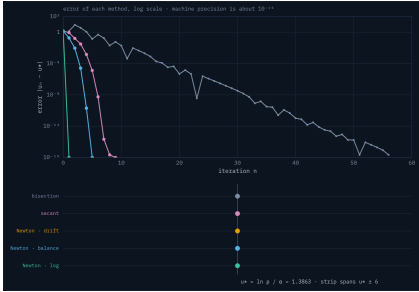
(a) **Simplex flow.** Training runs (streaks) flow across the three-outcome simplex to p^* (glowing dot); the dashed path is $p^*(\alpha)$.



(b) **Tetrahedron, 3-D.** Hundreds of four-outcome runs converge to p^* inside a rotating tetrahedron.



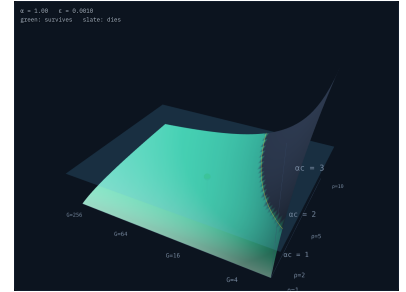
(c) **Amplification landscape, 3-D.** $\log |R(z)|$ of RK4; the crater below the plane is the stability region, the dot is $h\lambda$.



(d) **Root-finder race.** Error of five solvers against iteration, with each iterate's position below (Case 7).



(e) **Nested solver.** Five-outcome stationary shares, each outcome's threshold, and the outer residual per iteration.



(f) **Survival surface, 3-D.** α_c over (G, ρ) ; outcomes under the plane at the chosen α survive.

Figure 2: Six of the 19 instruments of the interactive dashboard, captured from the running application.
Table 4: Every Monte Carlo run. Rewards: “tie” (all equal), (4, 1), or (5, ..., 1) = (5, 4, 3, 2, 1); $\varepsilon = 10^{-3}$ throughout.

Case	K , rewards	α	G	h	steps	seeds
2a	2, tie	0	16	0.5	3k	36
2b	5, tie	0	16	0.5	6k	200
2c	2, tie	0	4–64	0.5	15k	80 per G
9a	5, tie	0–1	16	0.5	6k	100 per α
9b, 9c	2, (4, 1)	0.3–2	2–128	0.05	30k	16 per cell
11b	5, (5, ..., 1)	1	2–64	0.01	100k	16
14a	groups only	1	16	–	–	400k groups
16c	5, (5, ..., 1)	1	8–64	0.01	100k	32
18a, 18b	chains only	–	32	–	–	60k chains

Monte Carlo use the agent of Section 2.3.4, the others are deterministic tests. Table 5 lists every prediction with the case that tests it.

4.1 Validation of the stability law

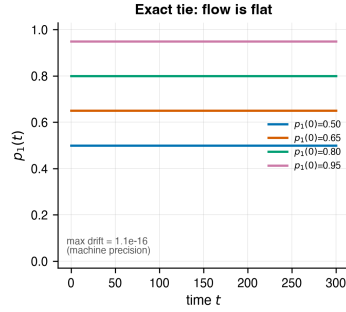
Key findings. The 18 cases support six conclusions.

- (1) **Collapse at equal rewards comes from sampling noise, not from the objective.** The exact flow does not move at a tie (1a); only the sampled agent collapses, onto a random answer (2a, 2b). With a reward gap, collapse time follows the exact law $53.89/\Delta$ (1c).

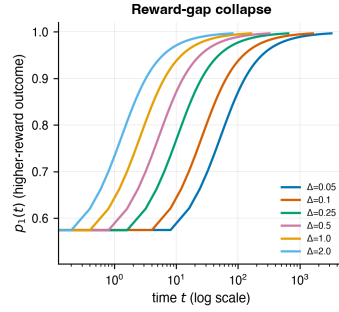
Prediction. Exact tie $\Rightarrow \dot{z} \equiv 0$: the flow cannot move. A gap $\Delta \Rightarrow$ collapse at $t_c = 53.89/\Delta$, from $u + \sinh u = \Delta t$.

Test. RK4, $h = 0.05$. Tie: four starts $p_1(0) \in \{0.5, 0.65, 0.8, 0.95\}$, $T = 300$. Gap: 12 values $\Delta \in [0.02, 4]$.

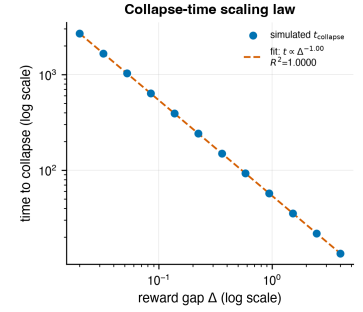
Result. Drift 1.1×10^{-16} (machine precision). Fit $t_c = 53.94 \Delta^{-1.00}$, $R^2 = 0.99999994$. ✓



(a) **Exact tie: the flow is flat.** Four RK4 runs from different starts stay perfectly flat: at a tie the drift is exactly zero.



(b) **Gap Δ : collapse.** p_1 over log time for six gaps Δ : every run collapses onto the better outcome, later for small Δ .



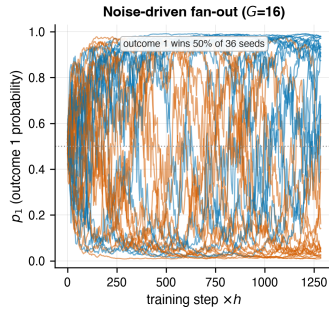
(c) **Collapse time vs. Δ .** Collapse time against Δ (log-log): the dots lie on the fitted line $53.94 \Delta^{-1.00}$.

Figure 3: Case 1: $\alpha = 0$ without noise: tie and gap. Validates Sec. 2.2.4, Eq. (6) · Exp. 1.

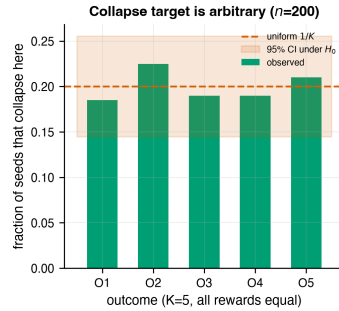
Prediction. A finite group breaks the tie at random and the multiplier p_i amplifies it: every seed collapses, onto each outcome with probability $1/K$.

Test. Symmetric start $z = 0$, $G = 16$, $h = 0.5$. 36 seeds ($K = 2$), 200 seeds ($K = 5$), 80 seeds per $G \in \{4, \dots, 64\}$.

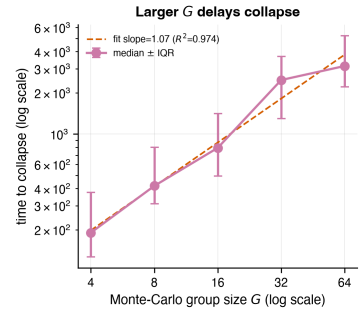
Result. Outcome 1 wins 50.0%. All $K = 5$ frequencies lie in the 95% band $[14.5, 25.5]\%$. Delay $t_c \propto G^{1.07}$. ✓



(a) **$K = 2$: noise breaks the tie.** 36 Monte Carlo runs from $p_1 = 0.5$: noise alone drives each run to 0 or 1, half of them each way.



(b) **$K = 5$: the winner is arbitrary.** Winner shares of 200 runs with $K = 5$ (bars) all sit inside the 95% band around $1/K$ (shaded).



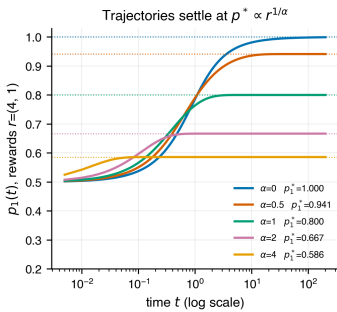
(c) **Larger G delays collapse.** Median collapse time (with IQR) grows with the group size, slope 1.07 on log-log axes.

Figure 4: Case 2: $\alpha = 0$ with sampling noise. Validates Sec. 2.2.4 · Exp. 1 · Monte Carlo.

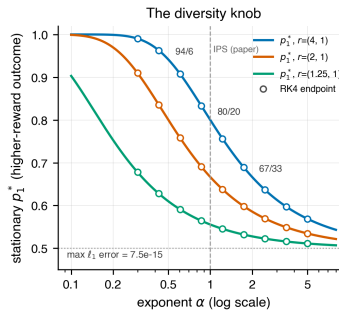
Prediction. $p_i^* = r_i^{1/\alpha} / \sum_k r_k^{1/\alpha}$ (Eq. 4). For $r = (4, 1)$: 94/6, 80/20 and 67/33 at $\alpha = 0.5, 1, 2$.

Test. RK4, $h = 0.005$, $T = 200$. 27 runs over $\alpha \in [0.3, 5]$ and three reward ratios; $K = 5$ with $r = (5, 4, 3, 2, 1)$.

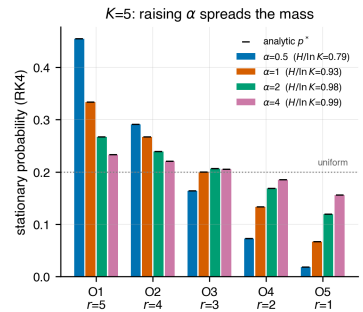
Result. Max ℓ_1 error 7.5×10^{-15} ($K = 2$) and 1.0×10^{-15} ($K = 5$). Entropy rises monotonically with α . ✓



(a) **Trajectories settle at p^* .** p_1 over time for $r = (4, 1)$ and $\alpha = 0$ to 4: each run settles at its own p^* (dashed).



(b) **p_1^* vs. α .** Final p_1 against α for three reward ratios: RK4 markers lie on the closed-form lines.



(c) **$K = 5$: stationary laws.** Stationary shares of five outcomes for several α : a larger α spreads the mass more evenly.

Figure 5: Case 3: Fixed point $p^* \propto r^{1/\alpha}$. Validates (S1) · Exp. 2.

Prediction. $\|p(t) - p^*\| \sim e^{-\lambda_{\min} t}$ with $\lambda_j = \alpha \|r\|_{1/\alpha} \mu_j$ (Eq. 5). At $\alpha = 0$ the decay is algebraic, $p_2 \approx 1/(2\Delta t)$.

Test. Least-squares slope of $\ln\|p - p^*\|_1$ in the window $[10^{-12}, 10^{-6}]$: 34 runs, $K = 2$ and 5.

Result. Rate error $\leq 1.7 \times 10^{-5}$ ($K = 2$). Tail slope -1.007 at $\alpha = 0$. At $\alpha = 0.1$: 3.899×10^{-4} vs. 3.906×10^{-4} predicted. ✓

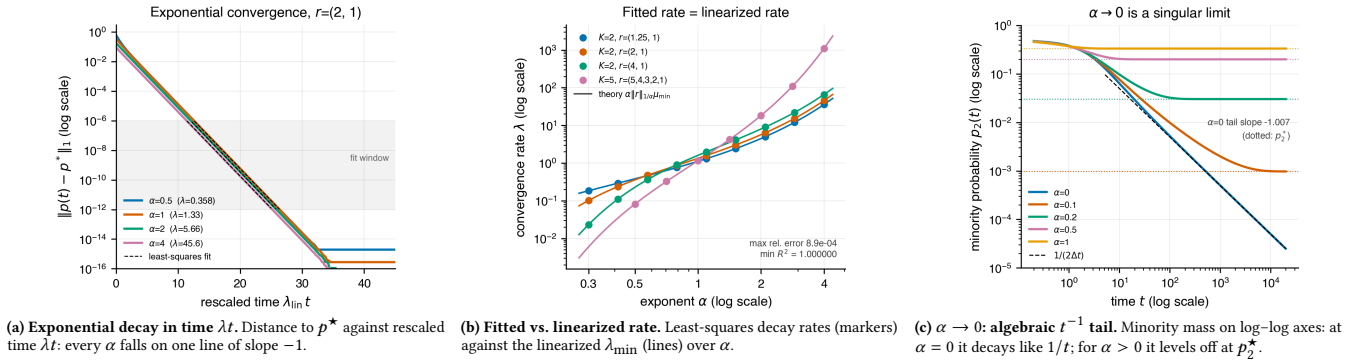


Figure 6: Case 4: Convergence rate and the $\alpha \rightarrow 0$ limit. Validates (S2) · Exp. 2.

Prediction. Explicit training is stable iff $h\lambda_{\max} < 2$ (Euler) or $h\lambda_{\max} < 2.7853$ (RK4). $\lambda_{\max}$ grows like K^α .

Test. Bisection (30 halvings) on the nonlinear update for the largest h at which a 10^{-4} perturbation of z^* shrinks; $\alpha \in [0.5, 4]$.

Result. Critical h matches theory to 2.6×10^{-4} (Euler) and 1.3×10^{-4} (RK4). $h = 0.05$ is unstable above $\alpha = 2.06$. ✓

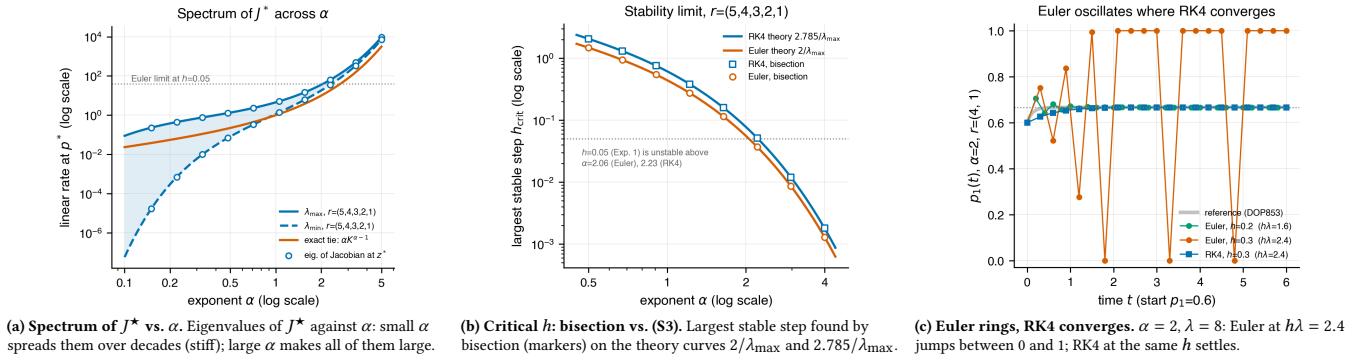


Figure 7: Case 5: Stable learning rate. Validates (S3) · Exp. 2.

Prediction. Global error $O(h)$ for Euler and $O(h^4)$ for RK4, down to the round-off floor.

Test. Error at $t = 15$ against scipy DOP853 ($\text{rtol} = 10^{-12}$) over a range of h ; production step $h = 0.05$.

Result. Observed orders 1.00 and 3.99. At $h = 0.05$ the error is 5.1×10^{-8} (RK4) and 1.4×10^{-4} (Euler). ✓

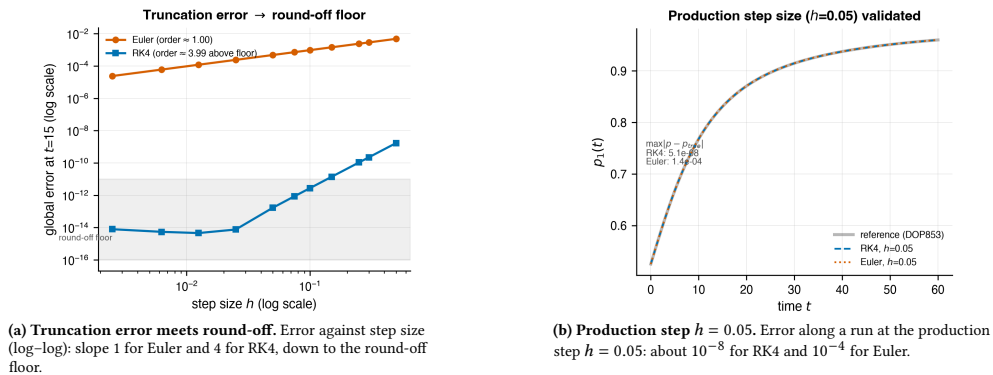


Figure 8: Case 6: Integrator accuracy. Validates solver orders · Exp. 1.

Prediction. Bisection is linear (ratio $\frac{1}{2}$), secant has order 1.618, Newton order 2. The log form is linear in u , so Newton is exact in one step.

Test. $\alpha = 1$, $r = (4, 1)$, root $u^* = \ln 4$ from $u_0 = 0$. Basin scan over $\alpha \in [0.2, 4] \times u_0 \in [-15, 15]$.

Result. Ratio 0.502; orders 1.61 and 2.00; log form in 1 step. Drift-Newton basin 14%, 17%, 100% at $\alpha = 0.5, 1, 2$. ✓

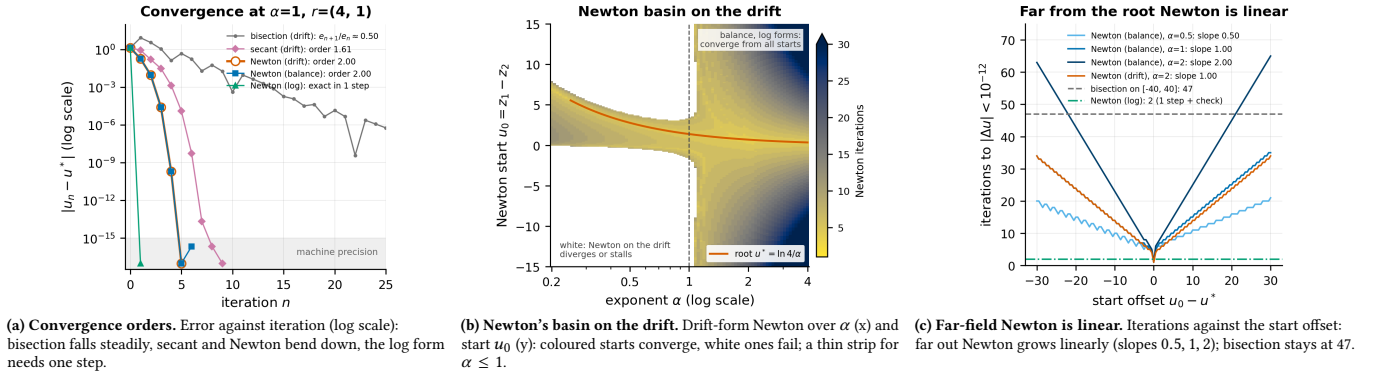


Figure 9: Case 7: Root-finding for $K = 2$. Validates (S1) directly · Exp. 3.

Prediction. With the gauge $z_K = 0$ Newton converges quadratically. At the root the balance and log Jacobians are $-\alpha \|r\|_1 / \alpha I$ and $-\alpha I$.

Test. Uniform start and 200 random starts $z \sim N(0, 3^2)$ for $\alpha \in [0.2, 4]$, with and without backtracking.

Result. Orders 1.98 (drift) and 1.94 (balance). Balance and log succeed from 100% of starts, drift from 0.5–6% for $\alpha < 1$. Condition number 1. ✓

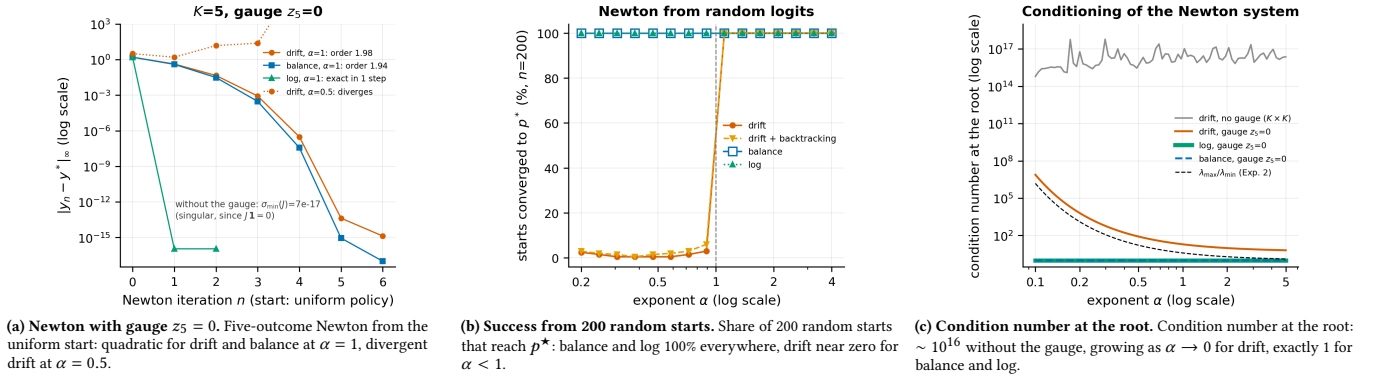


Figure 10: Case 8: Multivariate Newton for $K = 5$. Validates (S1), conditioning · Exp. 3.

Prediction. The minority survives iff $\alpha > \alpha_c = \ln(r_1/r_2) / \ln \min\{G, 1/\epsilon\}$. Any $\alpha > 0$ removes the collapse of Case 2.

Test. $K = 5$ tie: 100 seeds per α , $G = 16$, $h = 0.5$. $K = 2$, $r = (4, 1)$: 12 values of $\alpha \times G \in \{2, \dots, 128\}$, 16 seeds, 3×10^4 steps.

Result. Collapse 64% at $\alpha = 0$, none for $\alpha > 0$. |MC – mean field| ≤ 0.013 (≤ 0.0008 for $G \geq 32$). Boundary on α_c . ✓

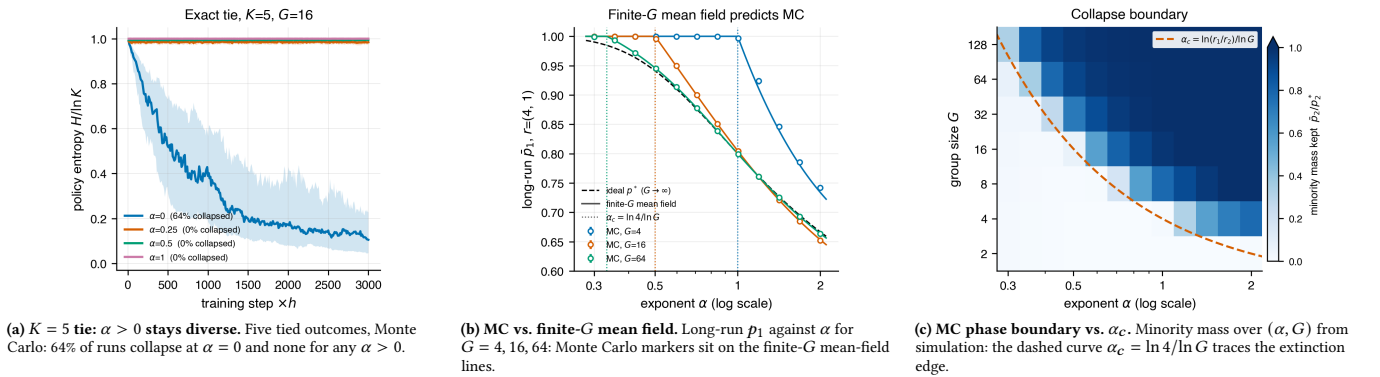


Figure 11: Case 9: Survival limit in the Monte Carlo agent. Validates (S4) · Exp. 2 · Monte Carlo.

Prediction. The sign of the margin $m = \alpha \ln \min\{G, 1/\varepsilon\} - \ln(r_1/r_2)$ alone decides survival; the clip caps the boost when $\varepsilon > 1/G$.

Test. Batched bisection on $51 \times 15 \times 5 \times 5 = 19,125$ points in $(\alpha, G, \varepsilon, r_1/r_2)$; every point re-solved by the margin-free scalar solver.

Result. 0 mismatches at the 19,120 points with $|m| > 10^{-9}$; the 5 points on $m = 0$ are collapsed, as required. Batched vs. scalar 9.1×10^{-13} ; balance residual $\leq 5.9 \times 10^{-15}$. ✓

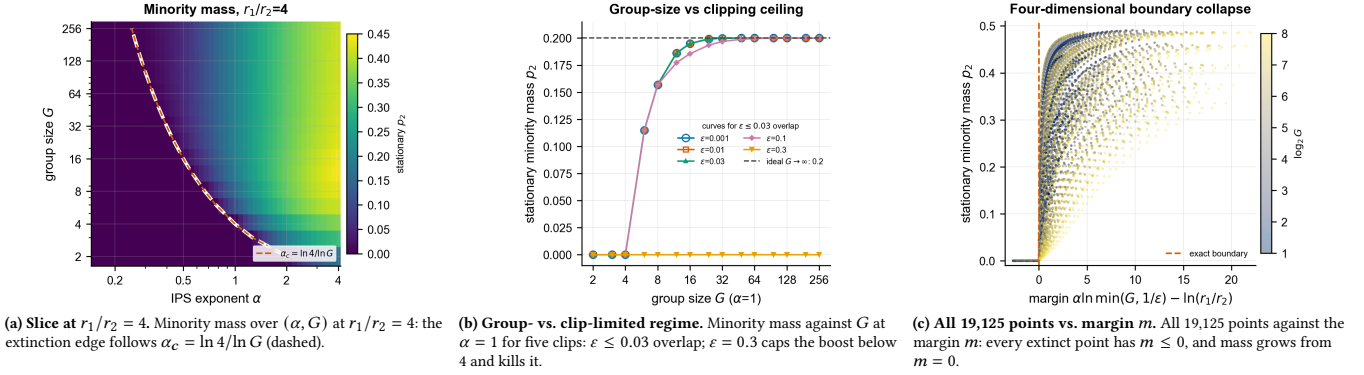


Figure 12: Case 10: Survival limit on a four-dimensional hypergrid. Validates (S4) · Exp. 4.

Prediction. $r_i w_G(p_i) = S$ has a unique nested root. Weaker outcomes need α above the pairwise threshold, and some are lost at every α .

Test. Nested bisection vs. safeguarded Newton, $G \in \{2, \dots, 256\}$. MC agent, $r = (5, 4, 3, 2, 1)$, $\alpha = 1$: 16 seeds, 10^5 steps, $h = 0.01$.

Result. Solvers agree to 9.8×10^{-14} ; MC vs. mean field ≤ 0.0049 . Runner-up $\alpha_c = K = 2$ formula to 5×10^{-10} ; weakest outcome needs 0.886. ✓

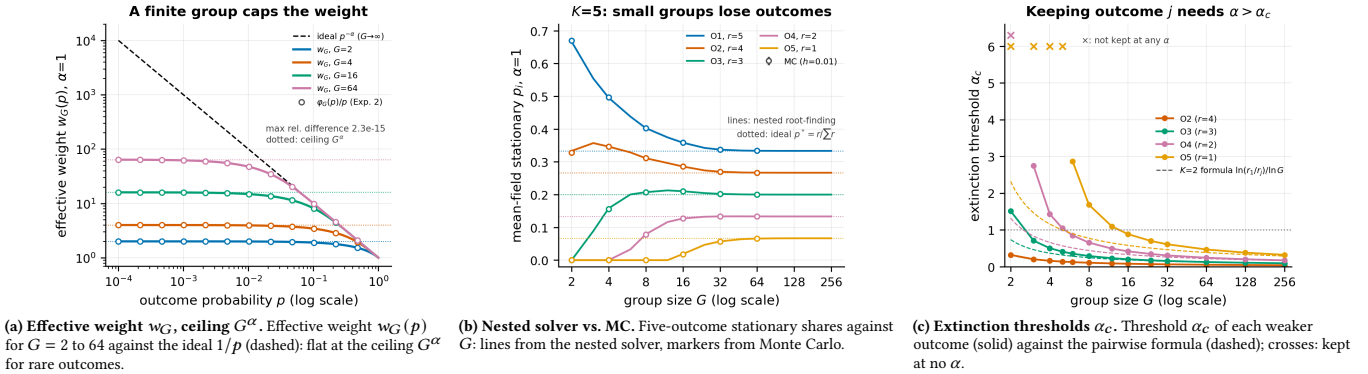


Figure 13: Case 11: Finite groups for $K = 5$. Validates Sec. 2.2.6 · Exp. 3 · Monte Carlo.

Prediction. Support grows monotonically with α and G . Full support needs a group that grows with K and with the reward spread.

Test. 765 KKT solves ($45 \times 17 G$) for $r = (5, 4, 3, 2, 1)$; exhaustive integer search in G for $K \leq 10$, spreads 1.25–32.

Result. Normalization error 6.0×10^{-12} , KKT residual 2.1×10^{-13} , no monotonicity violations. Minimum full-support G : 2 to 96. ✓

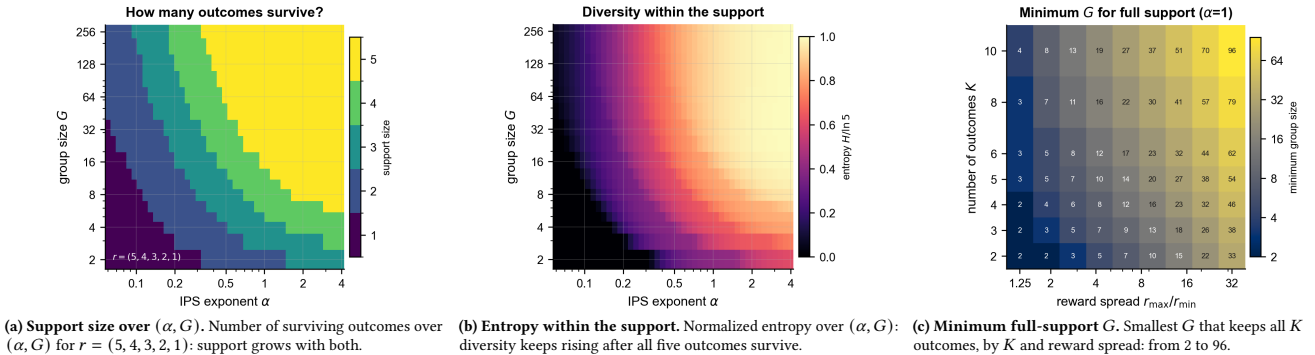


Figure 14: Case 12: General- K diversity atlas. Validates Sec. 2.2.6 · Exp. 4.

Prediction. Multilinear interpolation is exact at nodes, second order in smooth cells, and about first order in cells that contain a kink ($m = 0$ or any $\varepsilon = k/G$).

Test. 17^3 grid in $(\log \alpha, \log p, \log \varepsilon)$ at $G = 16$; 800 seeded off-grid direct solves; refinement $5 \rightarrow 9 \rightarrow 17$ points per axis.

Result. Node error 0. RMSE $2.8 \times 10^{-2} \rightarrow 5.2 \times 10^{-3}$. Orders 2.19, 1.86 in kink-free cells vs. 1.01, 1.21 in kinked cells. ✓

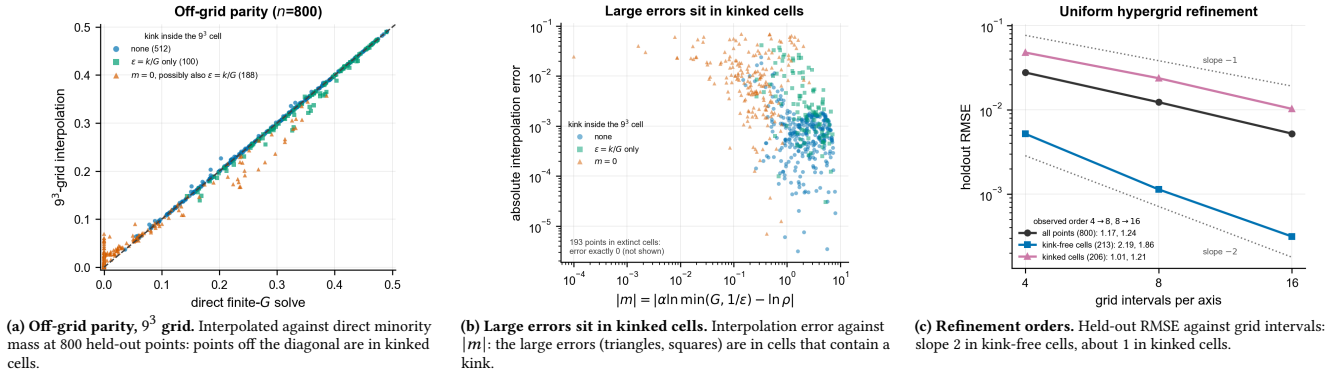


Figure 15: Case 13: The hypergrid as a surrogate. Validates interpolation error · Exp. 4.

Prediction. Relative bias $\frac{\alpha(\alpha+1)(1-p)}{2Gp}$, spread $\alpha\sqrt{\frac{1-p}{Gp}}$ (Eq. 9); the MSE-optimal clip is exactly $\varepsilon^* = p$.

Test. Exact binomial sums at $G = 16$, $\alpha = 1$; 400,000 Monte Carlo groups at each of 10 values of p ; golden-section search.

Result. Worst z-scores 2.20 (mean) and 2.46 (variance). Delta method valid for $Gp \gtrsim 20$. $\varepsilon^*/p = 1 \pm 7 \times 10^{-9}$. ✓

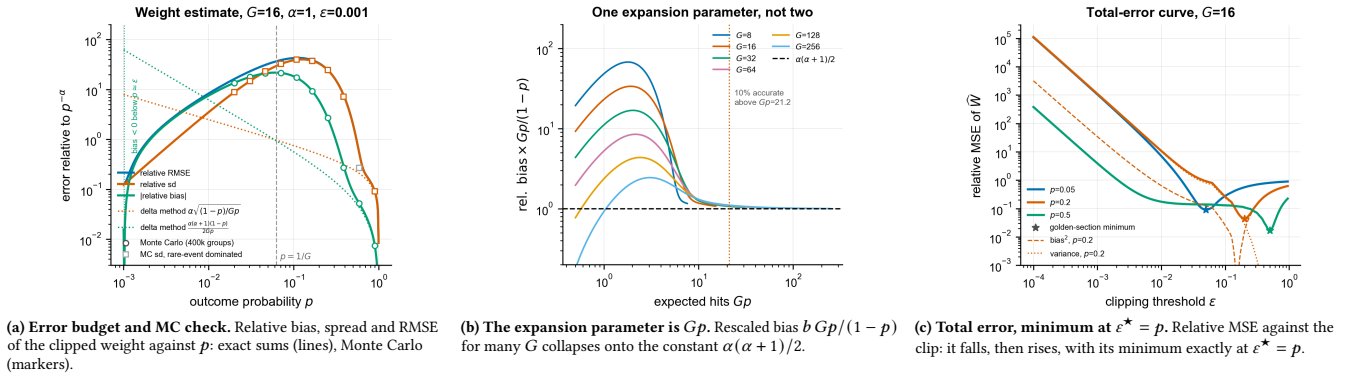


Figure 16: Case 14: How far is $\hat{p}$ from p ?. Validates Eq. (9), $\varepsilon^* = p$ · Exp. 5 · Monte Carlo.

Prediction. Richardson in $1/G$: bias order $1 \rightarrow 2$ at no leading variance cost, but naive weights go negative. An EMA cuts variance for free.

Test. Exact sums at $G = 16$, $p = 0.1$ (trade-off plane); bias fits in $\log G$ at $p = 0.1$ and 0.3 ; positivity-guarded split.

Result. EMA RMSE 0.077 vs. 42.6 for the paper's clip. Bias order $1.03 \rightarrow 2.10$. Naive singleton weight -486 to -276 . ✓

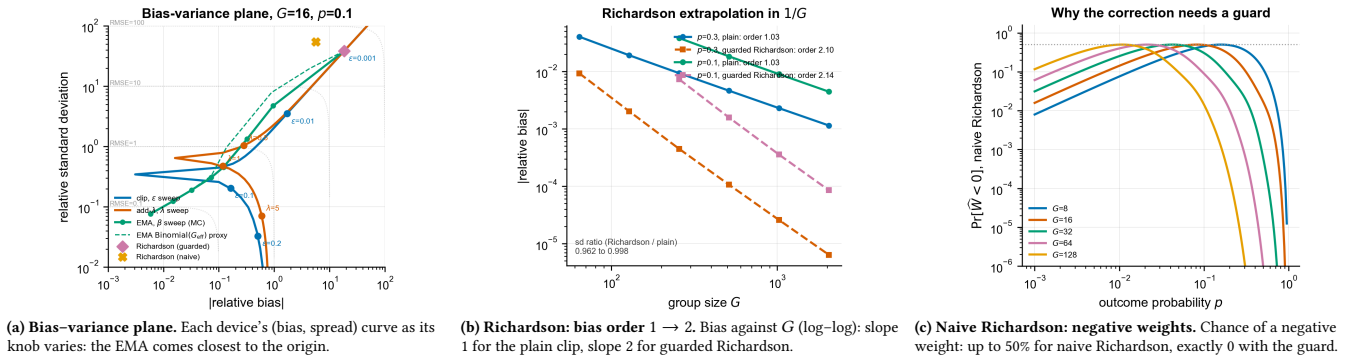


Figure 17: Case 15: Clipping, Laplace, Richardson and EMA. Validates Sec. 2.2.7 · Exp. 5.

Prediction. Survival iff $\omega(1)/\omega(G) > r_1/r_2$ for any rule. **Test.** Exact sums and bisection for α_c by rule. $K = 5$ MC training at $\alpha = 1$: 32 seeds, 10^5 steps, $h = 0.01$. **Result.** Range at $G = 16$: 16.0 (clip), 8.5 (Laplace), -472 (naive). α_c matches closed form to 7×10^{-11} . MC vs. mean field ≤ 0.0060 . ✓

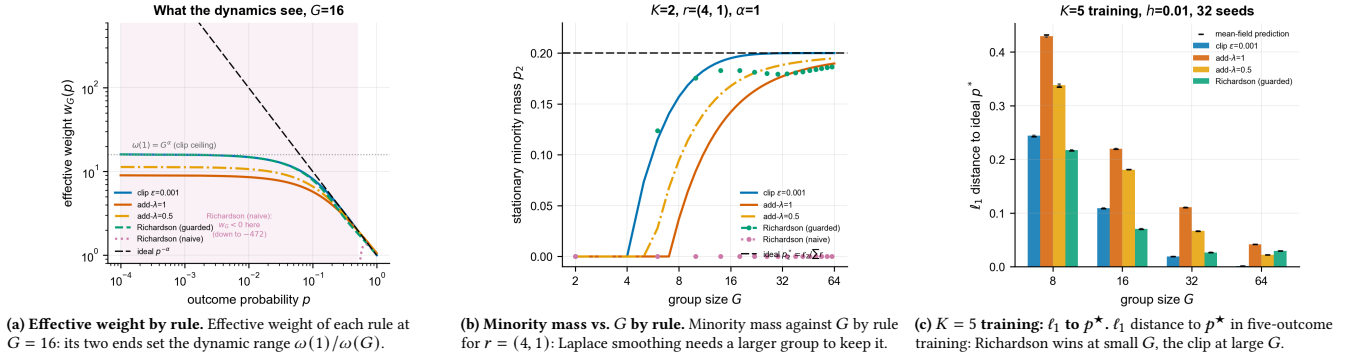


Figure 18: Case 16: Survival for any weight rule. Validates (S4), universal form · Exp. 5 · Monte Carlo.

Prediction. $D = \frac{\alpha(\alpha-1)(1-p)}{2Gp}$, and exactly $D = -(1-p)^G$ at $\alpha = 1$; the offset $c^* = \frac{1-\alpha}{2}$ restores second order. **Test.** Exact binomial sums for G up to 4096, $p \in [10^{-12}, 1]$, $\alpha \in [0.25, 3]$; order fits over $G \in [64, 2048]$. **Result.** The $\alpha = 1$ law holds to 4.0×10^{-11} . Coefficient $\alpha(\alpha-1)/2$ within 0.24%. Offset order 2.03–2.05. ✓

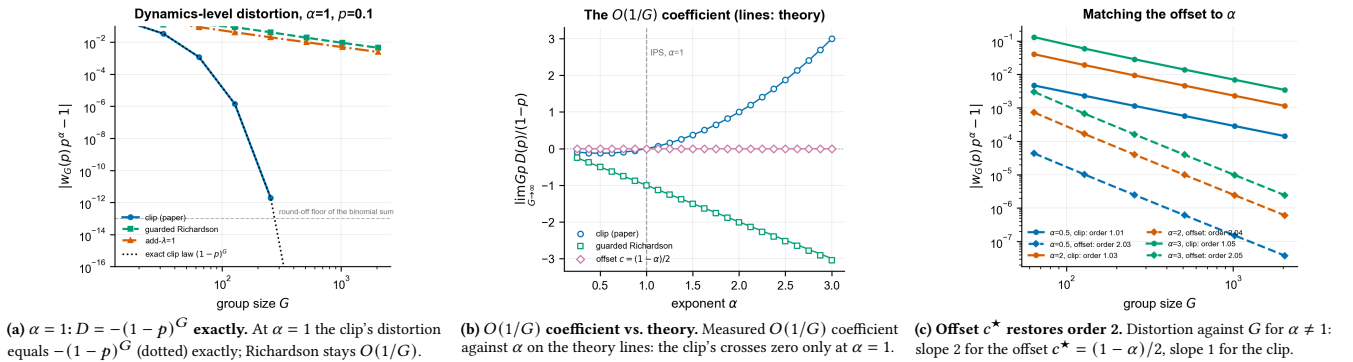


Figure 19: Case 17: What the training dynamics see. Validates (S5), Eqs. (10)–(11) · Exp. 5.

Prediction. Variance ratio $\beta/(2-\beta)$, i.e. $G_{\text{eff}} = G(2-\beta)/\beta$; $\log(1-\beta)\delta/\beta$; $s^2 + \kappa s + \kappa\lambda = 0$, fastest (2λ) at $\kappa = 4\lambda$. **Test.** 60,000 Monte Carlo chains, $G = 32, p = 0.3$; drift test with $\delta = 5 \times 10^{-4}$; eigenvalues by power method and QR. **Result.** Variance within 1.2%; lag z-score ≤ 1.40 ; $\beta^* = 0.051572$ (scan 0.051559); peak rate exactly $2.000 \times$ the plain rate. ✓

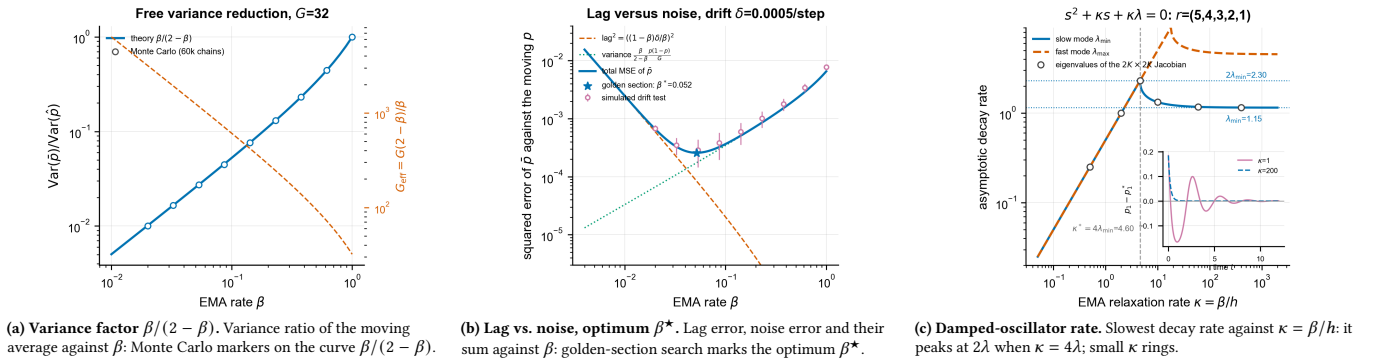


Figure 20: Case 18: Moving averages. Validates Eq. (12), G_{eff} · Exp. 5 · Monte Carlo.

Table 5: Theory vs. numerics: each derived formula, its measured agreement, and the case that shows it.

Prediction	Measured	Case
$\dot{z} \equiv 0$ at exact tie	drift 1.11×10^{-16}	1
$t_c = 53.89/\Delta$	$53.94 \Delta^{-1.00}$	1
noise collapse, $P = 1/K$	50.0%, in 95% band	2
$p^* \propto r^{1/\alpha}$ (S1)	ℓ_1 error 7.5×10^{-15}	3
$\lambda_{\min}$ (S2)	rel. error $\leq 1.7 \times 10^{-5}$	4
$p_2 \sim t^{-1}$ at $\alpha = 0$	slope -1.007	4
$h_c \lambda_{\max} = 2, 2.785$ (S3)	$2.6 \times 10^{-4}, 1.3 \times 10^{-4}$	5
Euler/RK4 order 1/4	1.00/3.99	6
Newton/secant order	2.00/1.61	7
MC vs. mean field	≤ 0.013	9
boundary $m = 0$ (S4)	0 mismatches off $m = 0$	10
runner-up $\alpha_c =$ (S4)	5.0×10^{-10}	11
$\varepsilon^* = p$	$\varepsilon^*/p = 1 \pm 7 \times 10^{-9}$	14
α_c via $\omega(1)/\omega(G)$	7.0×10^{-11}	16
exact w_G at $\alpha = 1$	4.0×10^{-11}	17
coefficient $\frac{\alpha(\alpha-1)}{2}$	within 0.24%	17
$G_{\text{eff}} = G(2 - \beta)/\beta$	within 1.2%	18
EMA peak rate 2λ	$2.000 \times$	18

- (2) **α is a working diversity knob.** Training ends at $p^* \propto r^{1/\alpha}$ to machine precision (3b), at the predicted rate (4b); any $\alpha > 0$ prevents the noise-driven collapse (9a).
- (3) **The safe learning rate is predictable.** The largest stable step sits on $2/\lambda_{\max}$ (Euler) and $2.785/\lambda_{\max}$ (RK4) (5b); since $\lambda_{\max}$ grows like K^α , a large α needs a much smaller step (5a).
- (4) **How the equation is written matters as much as the solver.** Every method finds the same end point, but Newton on the raw update fails from most starts when $\alpha \leq 1$ (7b, 8b), while the balance and log forms converge from every start (condition number 1, 8c).
- (5) **A finite group decides which answers survive.** The rule $\alpha > \ln(r_1/r_2)/\ln \min\{G, 1/\varepsilon\}$ holds in simulation (9c) and at all 19,125 hypergrid points (10c). At the paper’s own $\alpha = 1$, a group of $G = 4$ loses an answer worth a quarter of the best one ($p_1 = 0.9965$). With five answers, weak ones need a larger α than the two-answer rule says (0.886 instead of 0.580 at $G = 16, 11c$), and some cannot be kept at all when G is small. For $\alpha > 1$ a finite group even *over*-protects the minority: at $\alpha = 4$ its share peaks at 0.445 ($G = 12$), above the $G \rightarrow \infty$ value 0.414 (10a). A precomputed grid cannot replace this calculation near the boundary: even at 17^3 it gives 37 extinct held-out points a share above 10^{-3} (13b), so survival should be decided from the closed-form margin m .
- (6) **The paper’s estimator is better than it looks.** Its error as an estimate of $p^{-\alpha}$ is large (14a), but the part that moves the end point is exactly $-(1-p)^G$ at $\alpha = 1$ (17a). A “better” bias-corrected estimator therefore trains worse at large G (16c); Laplace smoothing always costs diversity (16b); and a moving average reduces noise without extra samples (15a, 18a).

Table 6: Base-paper claims and what our derivation and experiments add.

Base paper [3]	This work
Expected return collapses (Thm. 3.1)	Confirmed; collapse time $53.94 \Delta^{-1.00}$ matches the exact $53.89/\Delta$ (Case 1)
Collapse “even with equal rewards”	Deterministic flow is flat to 10^{-16} at a tie; collapse comes only from sampling noise (Cases 1–2)
IPS gives $p^* = r/\sum r$	Reproduced at $\alpha = 1$ to 7.5×10^{-15} , generalized to $p^* \propto r^{1/\alpha}$ (Case 3)
Global convergence, no rate	Rate $\lambda = \alpha \ r\ _{1/\alpha} \mu$ to 1.7×10^{-5} ; stable step $h < 2/\lambda_{\max}$ (Cases 4–5)
Clip ε prevents extreme weights	The boost is capped at $\min\{G, 1/\varepsilon\}^\alpha$, so survival needs $\alpha > \alpha_c$; at $\alpha = 1$, $G = 4$ loses an outcome worth $1/4$ ($p_1 = 0.9965$) (Cases 9–10)
Table 4: identical entries, read as bias–variance	For $\varepsilon < 1/G$ the clip never fires, so those entries must coincide; the MSE-optimal clip is $\varepsilon^* = p$, per outcome (Case 14)
Noisy $\hat{p}$ is a limitation	At $\alpha = 1$ the paper’s rule is exact up to $-(1-p)^G$; a bias-corrected estimator trains worse (Cases 16–17)

Design rule.

- (1) *Diversity*: choose α from $p^* \propto r^{1/\alpha}$ (S1).
- (2) *Survival*: take $G > (r_1/r_2)^{1/\alpha}$ and $\varepsilon < (r_2/r_1)^{1/\alpha}$ (S4); for $K > 2$, use the full-support G of Section 2.2.6.
- (3) *Stability*: keep the learning rate $h < 2/\lambda_{\max}$ (Euler) along the whole path (S2)–(S3).
- (4) *Estimator*: at $\alpha = 1$ the clipped rule is exact up to $-(1-p)^G$; for $\alpha \neq 1$ and large G , use the offset $c^* = (1-\alpha)/2$ (S5).

4.2 Comparison with the base paper

Table 6 sets our results against the base paper’s claims. Our test bed is the paper’s own theoretical abstraction, not its LLM and chemistry benchmarks, so the comparison concerns the mechanism and the weight rule; the ℓ_1 metric is the same, but the tasks differ.

4.3 Discussion

A formula for stable, diverse training. Box 1 turns into a practical design rule for α , G , ε and h , shown in the *Design rule* box. Each step is a closed-form quantity, and each was confirmed numerically (Cases 3, 10, 5, 17). The learning rate must respect the whole path, because the local rate during the transient can exceed the rate at p^* . The rule also explains the paper’s own clipping ablation: when $\varepsilon < 1/G$ the clip never fires, so those entries must coincide, and they do in the paper’s Table 4.

The formulation is part of the numerical method. Dividing the drift by p_i , which removes the same multiplier the base paper blames for collapse, turns Newton from locally convergent (14–17% basin) into globally convergent with condition number 1. Removing the multiplier helps the solver as well as the learning dynamics.

A better estimator can train worse. The dynamics see the size-biased weight (8), not the estimator’s MSE. At $\alpha = 1$ the paper’s rule is exact up to $-(1-p)^G$, and a bias correction destroys that cancellation. This is a two-sided result: Richardson helps at small G , where raw bias dominates, and hurts at large G .

5 Conclusion and Future Work

Key takeaways. Using only the methods of this course, we derived a closed-form stability law for α -generalized inverse probability scaling. It gives the fixed point, the convergence rates, the stable learning rate, and the universal finite-group survival limit $\omega(1)/\omega(G) > r_1/r_2$. Every part was verified numerically: ODE integration, root finding, least squares, a Monte Carlo agent, a 19,125-point hypergrid and exact error analysis all agree with the formulas, often to machine precision. The law turns a qualitative remedy for mode collapse into a quantitative design rule for α , G , ε and h , and it shows where the base paper’s choices are exactly right ($\alpha = 1$) and where they fail ($G \leq r_1/r_2$).

Limitations. Idealized model. All results are for the outcome-selection bandit with a group-REINFORCE update. GRPO’s per-group advantage standardization and PPO clipping are not modeled, so the conclusions concern the weight rule, not GRPO’s other machinery.

Mean-field limit. The finite- G theory is the $h \rightarrow 0$ limit of the sampled update. Monte Carlo gaps of up to 0.013 are finite- h bias, and they shrink with h as measured.

Numerical, not proven. The single sign change of $h_j(\alpha)$ for $K > 2$, and hence that the $\alpha \rightarrow \infty$ criterion decides whether a finite threshold exists, was checked on grids, not proven.

Limited reward profiles. The $K = 5$ results use one reward vector, $r = (5, 4, 3, 2, 1)$, plus geometric profiles in Exp. 4.

Exp. 1 figures. Experiment 1’s committed figures predate a fix to the sampled update (an absent outcome had received a spurious weight). A full re-run with the fix leaves every Experiment 1 conclusion unchanged.

Future work.

- Add GRPO’s group-standardized advantages and PPO clipping to the mean field, and test whether the survival limit still holds.
- Run the α knob and the group-size rule on the base paper’s own benchmarks (hyper-grid environment, HypoSpace) with neural policies.
- Choose G or α adaptively during training from the design rule, for example by raising G when an outcome’s margin approaches zero.
- Prove the single sign change of $h_j(\alpha)$ for $K > 2$, which we only checked on grids.
- Build a kink-aware hypergrid with nodes at $\varepsilon = k/G$, and use implicit or adaptive-step integrators for the stiff large- α regime.
- Extend IPS to actor-critic methods, left open by the base paper.

References

- [1] Charles R. Harris et al. 2020. Array programming with NumPy. *Nature* 585 (2020), 357–362.
- [2] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95.
- [3] Abhijeet Sinha, Sundari Elango, and Dianbo Liu. 2026. Expected Return Causes Outcome-Level Mode Collapse in Reinforcement Learning and How to Fix It with Inverse Probability Scaling. arXiv preprint arXiv:2601.21669. Base paper of this project.
- [4] Ahnaf Tahmid, Sadia Binte Sayed, Mohammad Raihan Rashid, Ruwad Naswan, and Md. Mehedi Hasan. 2026. alpha-ips-rl: Code for the numerical simulation of alpha-generalized inverse probability scaling. <https://github.com/rwd51/alpha-ips-rl>.
- [5] Pauli Virtanen et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272.

A Team Contributions

- **Ahnaf Tahmid** (2105041): Experiment 5, estimator bias and variance, bias-reduction devices, and their downstream cost (Cases 14–18).
- **Sadia Binte Sayed** (2105045): Experiment 3, root finding for the stationary point and the general- K finite-group mean field (Cases 7, 8, 11).
- **Mohammad Raihan Rashid** (2105046): Experiment 2, the α sweep, convergence rates, stiffness and step-size stability, and the Monte Carlo survival test (Cases 3, 4, 5, 9).
- **Ruwad Naswan** (2105051): Experiment 1, mode collapse at $\alpha = 0$, noise-driven collapse, and integrator validation (Cases 1, 2, 6).
- **Md. Mehedi Hasan** (2105052): Experiment 4, the four-dimensional finite-group hypergrid, the general- K atlas and the interpolation audit (Cases 10, 12, 13).

B Reproducibility

All code, the five experiment scripts, the unit tests and every figure are in a public GitHub repository, <https://github.com/rwd51/alpha-ips-rl>. To reproduce the results, download the repository (with `git`, or *Code* $\rightarrow$ *Download ZIP*) and run the scripts from its root folder (Python 3.10 or later, CPU only):

```
# 1. download the code from GitHub
git clone https://github.com/rwd51/alpha-ips-rl.git
cd alpha-ips-rl
# 2. install the dependencies
pip install numpy scipy matplotlib
# 3. run the five experiments
python3 experiments/exp1_collapse.py
python3 experiments/exp2_ips_alpha_sweep.py
python3 experiments/exp3_rootfinding.py
python3 experiments/exp4_hypergrid.py
python3 experiments/exp5_bias_variance.py
# 4. run the unit tests
python3 -m unittest discover -s tests -t .
# 5. dashboard (Node 20.19+), at localhost:5173
cd dashboard && npm install && npm run dev
```